

The Reactor Abstraction

How Hathor wraps Twisted's reactor in one typed, swappable seam — the ReactorProtocol, the global accessor, and the experimental asyncio backend.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 23 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Twisted reactor · ReactorProtocol · zope.interface · verifyObject · Global accessor · asyncio reactor · Adapter/shim

Table of Contents

Chapter 23 — The Reactor Abstraction	3
23.1 Localization	3
23.2 What it does and why it exists	4
23.3 The concepts it rests on	5
23.4 The code, walked	6
23.4.1 A tiny illustration first	6
23.4.2 ReactorProtocol — three slices joined into one type	6
23.4.3 get_global_reactor — the accessor	8
23.4.4 initialize_global_reactor — the installer, the verify, and the asyncio branch	8
23.4.5 The public surface	10
23.5 How it plugs into the lifecycle	11
The asyncio option, in current reality	11
Recap	12

Chapter 23 — The Reactor Abstraction

What you'll learn in this chapter

- Why Hathor wraps Twisted's reactor in its own tiny package instead of importing the global reactor everywhere.
- What `ReactorProtocol` is — a typed name for the slice of the reactor Hathor actually uses — and how it is stitched from three smaller protocols.
- How `initialize_global_reactor()` installs the reactor once at boot, and `get_global_reactor()` hands the same one to everyone afterward.
- What `verifyObject` checks before the reactor is handed out, and why that check sits next to a `cast`.
- What the experimental `--x-asyncio-reactor` flag does, why it is opt-in rather than the default, and the module-level-import footgun the code warns about.

This is the short companion to Chapter 16. There we taught the Twisted reactor, `Deferred`, `Protocol`, and `Factory` from the ground up. Here we look at one thin package built on top of that machinery: `hathor/reactor/`. The reactor itself is Twisted's; this chapter is about the small wrapper Hathor puts around it, and the two problems that wrapper solves. We recap Chapter 16's concepts only enough to follow along — the full treatment stays there.

23.1 Localization

`hathor/reactor/` is one of the smallest packages in the tree. It is an `__init__.py` plus two groups of files: one module for the type and one for the access functions.

```

hathor/
├─ builder/           ← Ch 24: assembles the node
├─ conf/             ← Ch 22: settings & profiles
├─ manager.py        ← HathorManager: the node's brain (Ch 29)
├─ p2p/              ← the peer-to-peer stack (Ch 34-35)
├─ reactor/          ← YOU ARE HERE
│   ├── __init__.py   (re-exports the 3 public names)
│   ├── reactor.py     (the accessor + the initializer)
│   ├── reactor_protocol.py (ReactorProtocol = the 3 below, joined)
│   ├── reactor_core_protocol.py (run/stop/callWhenRunning slice)
│   ├── reactor_time_protocol.py (callLater/seconds slice)
│   └── reactor_tcp_protocol.py (listenTCP/connectTCP slice)
├─ transaction/      ← the vertex model (Ch 25)
└─ ...

```

Six files, well under two hundred lines of real code between them. Yet almost every long-lived object in a running node holds a reference to whatever this package returns: the manager, the P2P connections, every timer, every scheduled retry. The reactor is the single thread on which the whole node lives (Ch 16), and `hathor/reactor/` is the one door through which the rest of the code reaches it.

Context. `hathor/reactor/` does not implement an event loop. It is a naming and access layer: it gives Twisted's loosely-typed reactor one clear type (`ReactorProtocol`), and it gives the whole program one function to install it (`initialize_global_reactor`) and one function to fetch it afterward (`get_global_reactor`). Everything downstream depends on this package instead of on `twisted.internet.reactor` directly.

23.2 What it does and why it exists

The package exists to solve two separate problems. Keep them apart in your head; they are why the code is shaped the way it is.

Problem 1 — the reactor has no useful type. When you write `from twisted.internet import reactor`, the object you get back is, as far as a type checker is concerned, barely typed.¹ Twisted's reactor is a single object that happens to provide more than a dozen interfaces² — one for core lifecycle (`run`, `stop`, `callWhenRunning`), one for timed calls (`callLater`), one for TCP (`listenTCP`, `connectTCP`), and so on. Code that calls `reactor.callLater(...)` has no compile-time guarantee the object even has that method. For a codebase that runs `mypy`,³ that is a real gap: bugs a type checker should catch slip through. Hathor closes it by declaring, in one place, exactly which capabilities it relies on, as one type a checker can follow.

Problem 2 — the node needs one shared reactor, reachable from anywhere. A reactor is, by nature, a singleton⁴ — a program has exactly one event loop. Dozens of objects, built at different times by the builder (Ch 24), all need to schedule work on that loop. Passing the reactor by hand through every constructor would be noisy and error-prone. So `hathor/reactor/` keeps the reactor in one module-level variable and exposes functions to install and reach it. This is the **global accessor** pattern (Ch 3): one well-known function, `get_global_reactor()`, that always returns the same object.

A useful way to see the whole package: it is a typed front door to a shared resource. The type is `ReactorProtocol`; the door is the accessor pair.

23.3 The concepts it rests on

This chapter leans on four ideas taught canonically elsewhere. Here are the reminders; follow the pointers if any feels unfamiliar.

Recap — the reactor / event loop (full treatment in Ch. 16 and Ch. 2). An event loop is a single thread that waits for events (a socket becomes readable, a timer fires) and runs the callback registered for each, one at a time, forever. Twisted's **reactor** is that loop. A network node uses one so it can juggle thousands of connections without a thread per connection. → full treatment in Ch. 16 (the framework) and Ch. 2 (the idea).

Recap — `zope.interface` and structural contracts (full treatment in Ch. 16 and Ch. 5). Twisted describes what the reactor can do with `zope.interface`⁵ interfaces — named bundles of methods, like `IReactorTime` ("can schedule timed calls"). An object that provides an interface promises those methods exist, and you can check that at runtime. This is the runtime-checked cousin of Python's `typing.Protocol` (Ch 5), the static-typing tool Hathor uses to mirror those interfaces. → full treatment in Ch. 16 (zope) and Ch. 5 (`Protocol`).

Recap — adapter / shim (full treatment in Ch. 3 and Ch. 4). An adapter is a thin layer that presents an existing thing through a different, more convenient interface, without changing the thing itself. `hathor/reactor/` is a shim: it neither replaces Twisted's reactor nor wraps its behavior — it only re-presents it under a clearer type and a single accessor. → full treatment in Ch. 3 (adapter) and Ch. 4 (shim).

Recap — singleton / global accessor (full treatment in Ch. 3). A singleton is a resource of which there is exactly one instance for the whole process. A global accessor is the function everyone calls to reach it. Together they avoid threading one shared object through every constructor by hand. Chapter 3 flagged this as a global-state smell; here it is unavoidable, because the event loop is singular. → full treatment in Ch. 3.

23.4 The code, walked

23.4.1 A tiny illustration first

Before the real code, a generic sketch of the move at the heart of this package: take a loosely-typed object and re-present it under a precise type.

Suppose a third-party library hands you a `connection` object whose type is `Any` — the type checker knows nothing about it. You know, from the docs, that it has `send()` and `close()`. You can declare a `Protocol` that names exactly those, then tell the type checker "treat this object as that":

```
from typing import Any, Protocol, cast

class Sendable(Protocol):
    def send(self, data: bytes) -> None: ...
    def close(self) -> None: ...

raw: Any = third_party.connect()      # type checker knows nothing
conn = cast(Sendable, raw)            # promise: it behaves like Sendable
conn.send(b'hello')                  # now type-checked
```

Nothing about `raw` changed at runtime. `cast`⁷ is a promise to the type checker, not a conversion. From here on, `conn.send` is checked and `conn.frobnicate()` is flagged as an error. That is precisely what `hathor/reactor/` does to Twisted's reactor — only the "promise" type is `ReactorProtocol`, and before the promise is made the code first verifies it is true (§23.4.4).

23.4.2 `ReactorProtocol` — three slices joined into one type

The type lives in `hathor/reactor/reactor_protocol.py`. It is built from three smaller protocols, one per Twisted interface Hathor uses (`reactor_protocol.py:22`):

```

from typing import Protocol

from hathor.reactor.reactor_core_protocol import ReactorCoreProtocol
from hathor.reactor.reactor_tcp_protocol import ReactorTCPProtocol
from hathor.reactor.reactor_time_protocol import ReactorTimeProtocol

class ReactorProtocol(
    ReactorCoreProtocol,
    ReactorTimeProtocol,
    ReactorTCPProtocol,
    Protocol,
):
    """
    A Python protocol that represents the intersection of Twisted's
    IReactorCore+IReactorTime+IReactorTCP interfaces.
    """
    pass

```

`ReactorProtocol` is a `typing.Protocol`⁶ — a structural type. Its body is `pass` because it adds nothing of its own; it exists purely to name the union of three capability slices. Each slice is its own small protocol that re-declares one Twisted interface so `mypy` can follow it:

- `ReactorCoreProtocol` — lifecycle: `run()`, `stop()`, `callWhenRunning()`, `addSystemEventTrigger()` (`reactor_core_protocol.py:38`).
- `ReactorTimeProtocol` — timers: `seconds()`, `callLater()`, `getDelayedCalls()` (`reactor_time_protocol.py:32`).
- `ReactorTCPProtocol` — TCP: `listenTCP()` (`reactor_tcp_protocol.py:33`), `connectTCP()` (`:43`).

Joining several contracts into one type like this gives an **intersection type**¹⁰: an object is a `ReactorProtocol` only if it provides all three slices at once. The class docstring says exactly that — it is "the intersection of Twisted's `IReactorCore` + `IReactorTime` + `IReactorTCP` interfaces."

Why split into three sub-protocols rather than list every method in one class? Because each slice mirrors one Twisted interface, so the code that verifies the real reactor (§23.4.4) can check one interface against one slice. The full Twisted reactor provides much more (UDP, SSL, threads, subprocesses); Hathor names only this slice. If a future need arises — say, UDP — the fix is to add a fourth sub-protocol and a fourth base, in this one place, and the checker will then permit `reactor.listenUDP(...)` everywhere.

Why not just type things as "the whole reactor"? Because Twisted exposes no single class that means "the reactor"; it ships a bag of interfaces and one concrete object that happens to satisfy many of them. `ReactorProtocol` is Hathor's answer to the missing name.

23.4.3 `get_global_reactor` — the accessor

The accessor and initializer both live in `hathor/reactor/reactor.py`. The module holds one piece of state (`reactor.py:26`):

```
# Internal variable that should NOT be accessed directly.
_reactor: ReactorProtocol | None = None
```

That module-level `_reactor` is the single slot the whole program shares.

`get_global_reactor()` (`reactor.py:29`) reads it:

```
def get_global_reactor() -> ReactorProtocol:
    global _reactor

    if _reactor is None:
        raise Exception('The reactor is not initialized. Use `initialize_global_reactor()`.')

    return _reactor
```

Read it top to bottom. If a reactor has been installed, return it — the common case, hit thousands of times. If the slot is empty, it does **not** silently invent one; it raises. That strictness is deliberate. There is exactly one correct moment to choose and install the reactor — early in boot, where the asyncio flag is known (§23.5). If

`get_global_reactor()` quietly installed a default reactor on first use, an accidental early call would lock in the wrong reactor and the `--x-asyncio-reactor` choice could never take effect. Raising turns that ordering mistake into a loud, named failure instead of a silent wrong default.

The docstring carries a second rule (`reactor.py:34`): this function "must NOT be called in the module-level, only inside other functions." The reason is the same import-time footgun we meet fully in §23.4.4 — touching the reactor while a module is being imported can install it before the initializer runs.

23.4.4 `initialize_global_reactor` — the installer, the verify, and the asyncio branch

The accessor reads the slot. The initializer fills it, and is the only place that gets to choose which reactor goes in. It is defined at `reactor.py:44`. Take it in three parts.

Part 1 — the already-installed guard.

```
def initialize_global_reactor(*, use_asyncio_reactor: bool = False) -> ReactorProtocol:
    global _reactor

    if _reactor is not None:
        log = logger.new()
        log.warn('The reactor has already been initialized. Use `get_global_reactor()`.')
        return _reactor
```

If the slot is already full (`reactor.py:51`), the function does not raise — it logs a warning and returns the existing reactor. The expectation is that this function is called exactly once; a second call is treated as a caller mistake worth a log line, not a crash. (Note the asymmetry with `get_global_reactor` , which raises when the slot is empty. Empty-on-read is a real bug; full-on-init is a harmless redundancy.)

Part 2 — the asyncio branch. When `use_asyncio_reactor` is true (`reactor.py:56`):

```
if use_asyncio_reactor:
    import asyncio

    from twisted.internet import asyncioreactor
    from twisted.internet.error import ReactorAlreadyInstalledError

    try:
        loop = asyncio.get_running_loop()
    except RuntimeError:
        loop = asyncio.new_event_loop()
        asyncio.set_event_loop(loop)

    try:
        asyncioreactor.install(loop)
    except ReactorAlreadyInstalledError as e:
        msg = (
            "There's a Twisted reactor installed already. It's probably the default one, installed
            "one of our imports. This can happen, for example, if we import from the hathon m
            "entrypoint-level, like in CLI tools other than `RunNode`. ..."
        )
        raise Exception(msg) from e
```

The branch reuses the current asyncio event loop if one is already running, otherwise creates and registers a fresh one (`reactor.py:62–66`). It then asks Twisted to install its `asyncioreactor` backed by that loop (`reactor.py:69`). The effect: Twisted's reactor and Python's `asyncio` share a single underlying loop, so code written against either can run together.

The `try/except` around `asyncioreactor.install` (`reactor.py:70`) catches a Twisted error, `ReactorAlreadyInstalledError`,⁹ and re-raises it with a pointed explanation. That message names the real footgun: importing anything from the `hathon` module at entrypoint level — as a CLI tool other than `RunNode` might — can install the **default** Twisted reactor indirectly, before this function runs. Once the default reactor is in, the asyncio install fails. The verbose message exists because this is a confusing failure to debug from the stack trace alone.

The ordering rule, stated plainly. Both `initialize_global_reactor` and `get_global_reactor` carry the docstring warning "must NOT be called in the module-level, only inside other functions" (`reactor.py:34` , `:47`). Twisted installs a reactor the first time `twisted.internet.reactor` is imported. If any module-level import triggers that before the initializer chooses, the asyncio branch can no longer take effect. This is why `run_node` calls the initializer from inside `prepare()` (`run_node.py:202`), not at the top of the file.

Part 3 — verify, then cast. Whether or not the asyncio branch ran, the function finishes the same way (`reactor.py:80`):

```
from twisted.internet import reactor as twisted_reactor

assert verifyObject(IReactorTime, twisted_reactor) is True
assert verifyObject(IReactorCore, twisted_reactor) is True
assert verifyObject(IReactorTCP, twisted_reactor) is True

# We cast to ReactorProtocol, our own type that stubs the necessary
# Twisted zope interfaces, to aid typing.
_reactor = cast(ReactorProtocol, twisted_reactor)
return _reactor
```

It imports whichever reactor is now current, then runs `verifyObject`⁸ three times (`reactor.py:82–84`) — once per Twisted interface that `ReactorProtocol` mirrors. `verifyObject` is a runtime check: it confirms the real reactor genuinely provides every method of `IReactorTime` , `IReactorCore` , and `IReactorTCP` . Only after those pass does the code `cast` the reactor to `ReactorProtocol` (`reactor.py:87`) and store it.

This pairing is the whole point of the abstraction in miniature. The `cast` is a promise to the static type checker (§23.4.1); the `verifyObject` asserts the runtime truth behind that promise. Recall from §23.4.1 that a bare `cast` is unchecked — it would happily lie. The three asserts make sure the lie is actually true before it is told, so a reactor that silently lacked, say, `connectTCP` would fail loudly here at boot rather than mysteriously much later.

23.4.5 The public surface

`hathor/reactor/__init__.py` re-exports the three public names — `initialize_global_reactor` , `get_global_reactor` , and `ReactorProtocol` (`__init__.py:18–22`). The rest of the codebase imports from `hathor.reactor` , never from `twisted.internet` . That one rule is what keeps the seam intact: change the wrapper once, and every consumer follows.

23.5 How it plugs into the lifecycle

Recall the boot skeleton from the Orientation chapter (Ch 0, §0.3, Act I). Step 4 is "**The reactor is initialized** — the event loop is created and installed as the global reactor." That step is a single call into this package.

Following the order in `run_node.py`'s `prepare()` method:

1. Settings are loaded and the network profile is selected (Ch 22).
2. Runtime preconditions are validated — `validate_args()`, `check_unsafe_arguments()`, `check_python_version()` (`run_node.py:197–199`).
3. **The reactor is installed:**
`initialize_global_reactor(use_asyncio_reactor=self._args.x_asyncio_reactor)` (`run_node.py:202`). The chosen reactor is stored on `self.reactor` (`:203`) and in the package's module slot.
4. Hand off to the builder (Ch 24):
`CliBuilder(self._args).create_manager(reactor)` (`run_node.py:208–210`). Every object that needs to schedule work is handed `self.reactor` or reaches it through `get_global_reactor()`.
5. At the very end of boot, `self.reactor.run()` (`run_node.py:592`) starts the loop. That call does not return until the node shuts down — it is Act II.

This placement is why the chapter sits where it does. Settings (Ch 22) decide what to build; the reactor (this chapter) provides the loop everything will run on; the builder (Ch 24) does the building. The reactor must exist before the builder runs, because the objects the builder creates capture a reference to it.

The asyncio option, in current reality

The `--x-asyncio-reactor` flag is defined at `run_node.py:161` with the help text "Use asyncio reactor instead of Twisted's default." Two facts about its current state, both visible in the code:

- **The `x-` prefix marks it experimental.** In this codebase the `x-` / `--x-` convention flags unstable, opt-in features — `--x-asyncio-reactor` sits in the same `UNSAFE_ARGUMENTS` list as `--x-ipython-kernel` (`run_node.py:58–59`). The flag uses `action='store_true'`, so it defaults to `False`.
- **The default is the standard Twisted reactor.** When the flag is off, `initialize_global_reactor` skips the asyncio branch entirely and installs Twisted's ordinary global reactor. You opt into the asyncio-backed reactor only when you need Twisted and `asyncio` code to interoperate on one loop.

Treating asyncio as opt-in is deliberate. The standard reactor is the well-tested production path; swapping the event-loop backend is the kind of change you do not want every node to take by default. The seam exists so the option can be exercised — not so it is the norm. The deeper "why Twisted, not asyncio" trade-off is argued in Ch 16, §16.7.

Recap

Fact	Where
The package is a typed seam over Twisted's reactor, not an event loop of its own	<code>hathor/reactor/</code>
<code>ReactorProtocol</code> = intersection of three slice-protocols mirroring <code>IReactorCore</code> / <code>IReactorTime</code> / <code>IReactorTCP</code>	<code>reactor_protocol.py:22</code>
The three slices declare the methods Hathor uses	<code>reactor_core_protocol.py:38</code> , <code>reactor_time_protocol.py:32</code> , <code>reactor_tcp_protocol.py:33</code>
One shared module-level slot holds the reactor	<code>reactor.py:26</code>
<code>get_global_reactor()</code> returns it; raises if not yet initialized	<code>reactor.py:29</code> , <code>:38</code>
<code>initialize_global_reactor(...)</code> installs it; a second call warns and returns the existing one	<code>reactor.py:44</code> , <code>:51</code>
The asyncio backend is installed before the reactor import; an indirect default-reactor install is caught and re-explained	<code>reactor.py:56–78</code>
<code>verifyObject</code> checks the three interfaces at runtime, then <code>cast</code> makes the static promise	<code>reactor.py:82–87</code>
Both accessor and initializer must not be called at module level	<code>reactor.py:34</code> , <code>:47</code>
Boot installs the reactor in <code>prepare()</code> , before the builder	<code>run_node.py:202</code>
<code>--x-asyncio-reactor</code> is experimental and off by default	<code>run_node.py:161</code> , <code>:58</code>
<code>self.reactor.run()</code> is the last line of boot	<code>run_node.py:592</code>

The reactor abstraction earns its keep with very little code: one type to make the reactor legible to `mypy`, one accessor so the whole node shares a single loop, and one initializer that picks the backend, verifies it, and installs it once. Hold onto two facts as you move on — every long-lived object you meet from here holds a reactor reference, and that reference is always handed out through this package. Next, **Chapter 24** turns to the builder: the composition root that constructs those objects and threads the freshly-installed reactor through all of them.

1. Twisted predates Python's modern type system, so its reactor is exported as a plain object satisfying many interfaces rather than as one statically-typed class. Without help, a checker treats most accesses on it as untyped. ↩
2. An interface is a named list of methods an object promises to provide — a contract. It says nothing about implementation, only about the shape of the object. ↩
3. `mypy` is a static type checker for Python: it reads type annotations and flags mismatches before the program runs. `hathor-core` runs it in CI (Ch 20), so untyped objects weaken its coverage. ↩
4. A singleton is a resource of which exactly one instance exists per process. An event loop is inherently one-per-process, which makes the reactor a natural singleton. Full treatment in Ch. 3. ↩
5. `zope.interface` is the library Twisted uses to declare and check interfaces at runtime. Its interface names conventionally start with a capital `I` (`IReactorTime`). It is the runtime-checked cousin of `typing.Protocol`. ↩
6. Python's `typing.Protocol` enables structural typing: any object with the required methods satisfies the protocol, with no need to inherit from it. It is a static-typing tool, checked by `mypy`, and is unrelated to Twisted's connection-handler `Protocol` class. Full treatment in Ch. 5. ↩
7. `typing.cast(T, x)` tells the type checker "treat `x` as type `T`." It performs no runtime check or conversion; it only changes how the checker reasons about the value — which is why Hathor pairs it with a runtime `verifyObject` check. ↩
8. `zope.interface.verify.verifyObject(Interface, obj)` checks at runtime that `obj` actually provides every method the interface declares, raising if not. Hathor uses it to confirm the real reactor matches `ReactorProtocol` before casting to that type. ↩
9. Twisted's own `ReactorAlreadyInstalledError` is raised by `asyncioreactor.install` when a reactor is already in place. Hathor catches it only inside the `asyncio` branch, to replace the bare error with a message explaining the likely cause (an indirect default-reactor install at import time). ↩
10. An intersection type describes a value that satisfies several contracts at once. Python has no dedicated syntax for it, so the idiom is to define a type that inherits from each contract, as `ReactorProtocol` does with its three slice-protocols. ↩